ARTICLE

Reallocating a detector's output budget is positional: it replaces the worst image, and only when that image is quota-limited

Yuxuan Xia (1), Ziming Zeng (1,*)

1. Shenzhen Polytechnic University, Shenzhen, China

*Corresponding author: Ziming Zeng, zzm1983@szpu.edu.cn

First author: Yuxuan Xia, xiayuxuan@mail.szpu.edu.cn

Keywords: object detection; output budget; budget reallocation; worst-case recall; detection evaluation protocol; VisDrone

Abstract

Object detectors emit a bounded number of boxes per image, and that bound is a fixed constant: COCO sweeps maxDets = [1, 10, 100], LVIS fixes max_dets = 300. Images differ greatly in count, so a fixed bound is slack on sparse images and binding on dense ones. Given a fixed total of boxes, does redistributing them between images help? We show that it is a positional intervention rather than a performance one. On validation, reallocating raises worst-image recall by 5.3 to 10.9 points across three training seeds and both heads, but total recall changes by at most 1.41 points and mean per-image recall falls by 6.1 to 6.9 points. The gain is positional: it changes which image is worst rather than lifting every image, and it improves some images while degrading others. When that image is quota-limited the reallocation replaces it; when it is detection-limited it keeps zero recall at any quota and the gain vanishes on test-dev for both heads. The cost is sharply graded: the sparsest 5% of images lose 24 points, and in the condition tabulated here every band's change is bounded away from zero. Once the budget exceeds the label-count cap on every image, the two allocations become the same vector and the comparison is vacuous.

Introduction

A detector's output budget is the largest number of boxes it may emit for an image. It is rarely treated as a scientific choice. The COCO evaluation protocol sweeps maxDets = [1, 10, 100] [1]; the LVIS protocol fixes max_dets = 300 [2]; the reference implementations in common use default to 300. In each case a single constant is applied to every image in the benchmark, whatever those images contain.

That constant is not innocuous, and the observation is not new. Dave et al. [3] raised the detections-per-image limit on LVIS from 300 to 10,000 and reported that mean average precision rises from 22.6 to 26.1, with the average precision for rare categories rising from 12.6 to 19.8. The same intervention leaves COCO essentially unchanged at 37.4 to 37.5, which they attribute to COCO's smaller vocabulary. The first-place entry in the 2021 LVIS challenge reports a consistent effect on a different metric, with boundary AP rising from 44.1 to 45.3 as the per-image limit increases from 300 to 1,000, and notes that the competition rules imposed no limit at all [4]. Oksuz et al. [5] report from a third direction that localisation recall precision responds to detections-per-image, although in their setting average precision and oLRP move in opposite directions, so the two metrics do not agree on the sign of the change. Ultralytics issue #25992 [6] records an aggregate ceiling of the same kind: at the default limit, the fraction of annotated objects recoverable by construction is 23.6% on the benchmark discussed there.

Those reports are consistent with a fixed output budget being able to bind. All of them act by raising the limit, which gives every image more boxes. None asks what follows when the limit is

genuinely fixed, which is the situation in any deployed system with a hard quota. A concrete case is a downlink that can carry a fixed number of boxes per frame for a whole camera array, so that boxes spent on one camera are boxes denied to another. If the budget is fixed and the constraint binds, the distribution across images becomes a free parameter: the same total number of boxes can be spread evenly, concentrated on dense images, or allocated in any other way. Whether that freedom is worth exercising is an open question. The evaluation literature treats the output budget as a protocol constant rather than an allocatable resource, and the allocation literature that would supply the machinery addresses other budgets: compute or patch selection rather than emitted boxes [7], or a decision-theoretic value of perception compute under a budget-constrained oracle with uniform inference as the baseline [8]. Two lines do reallocate a per-sample capacity, but of computation rather than emitted boxes: content-aware detectors give each image a data-dependent compute budget [9], and computation reallocation moves backbone FLOPs across resolutions [10]. In retrieval and recommendation, a fixed output window has been formalised as a deterministic ceiling on achievable ranking quality [11, 12], but there the window is per query or per user, leaving no room for redistribution.

We therefore ask a narrow question. Given a fixed total number of boxes to distribute across an image set, does reallocating it change the worst-image recall, and what does the change cost? We answer with a measurement rather than a method. The proportional allocation we compare against is the classical max-min fair one, obtained by integer water-filling, and we claim nothing new for it [13, 14]. What we contribute is the magnitude of the effect on a real detector, the distribution of its cost across the images, and the regime in which the comparison is vacuous.

We evaluate this on VisDrone2019-DET [15] with YOLO26n [16], whose two prediction heads give two settings within one detector, and we train three seeds. Reallocating a fixed total budget raises worst-image recall by 5.3 to 10.9 points on validation, but not on the held-out test split. Three results constrain how that gain should be read.

First, the intervention is positional rather than absolute. It changes which image attains the minimum; some images gain, others lose. When we trace the identity of the worst image under both allocations, the uniform allocation consistently pins a dense image as the worst case, and reallocation replaces it with a different one. Mean per-image recall therefore falls even as the minimum rises.

Second, the total box count is conserved and the cost is graded. Total recall changes by at most 1.41 points, because when the budget binds on every image the number of boxes that can be spent is fixed by the budget. The cost falls unevenly: the sparsest 5% of images lose 24.2 points while the densest quartile gains 10.1, and mean per-image recall falls by 6.1 to 6.9 points. A practitioner who adopts per-image allocation to protect the worst case will redistribute recall towards images that already contained the most objects.

Third, the boundary of the effect is structural, not empirical. Once the budget exceeds the total number of boxes the detector can emit, the two allocations coincide exactly, and we verify this with an exact comparison of the two quota vectors. Any "zero gain" measured there is an identity, and we report it as such rather than as evidence about the allocation.

This work sits between three literatures that do not meet. Detection evaluation treats the per-image output limit as a protocol constant, and the reports that show it can bind act by raising it, which gives every image more boxes rather than moving boxes between images [3, 4, 5]. Resource-allocation work in detection budgets compute or input regions rather than emitted outputs, and has already argued that allocation should be judged against a budget-constrained oracle with uniform inference as a baseline [7, 8]. Information retrieval and recommendation formalise a fixed output window as a hard ceiling on achievable ranking quality [11, 12], but the window there is fixed per query or per user, so no redistribution is possible. What has been missing is a measurement of what redistribution actually buys, and at whose expense.

Two things follow for practice. First, worst-image recall is an extreme order statistic governed by whichever image happens to be worst, so treating it as a deployment objective moves the loss onto the images with the fewest objects rather than reducing it. Second, before per-image allocation is adopted, the budget should be checked against what the detector can actually emit: if the budget is not binding, the allocation question does not arise, and a comparison run there will return zero for structural reasons. We report these results for one detector on one benchmark, and we state the scope explicitly rather than generalising.

Results

R0. Output-budget utilisation varies by up to 8.5-fold across detection benchmarks

A detector's output budget is the maximum number of boxes it may emit per image. In practice this budget is a fixed constant: the COCO evaluation protocol sweeps maxDets = [1, 10, 100] [1], the LVIS protocol fixes max_dets = 300 [2], and the Ultralytics YOLO implementation defaults to max_det = 300 [16]. Whether that is reasonable depends on n_i, because a fixed budget binds only on images whose count exceeds it.

We therefore computed, for five benchmark splits, three quantities that depend only on the label distribution and not on any model: the mean per-image object count, the budget utilisation util(B) = Σ_i min(B, n_i) / (B·N), and the fraction of images whose count exceeds the budget, frac(n_i > B). Table 1 reports them and Figure 1 shows them as curves.

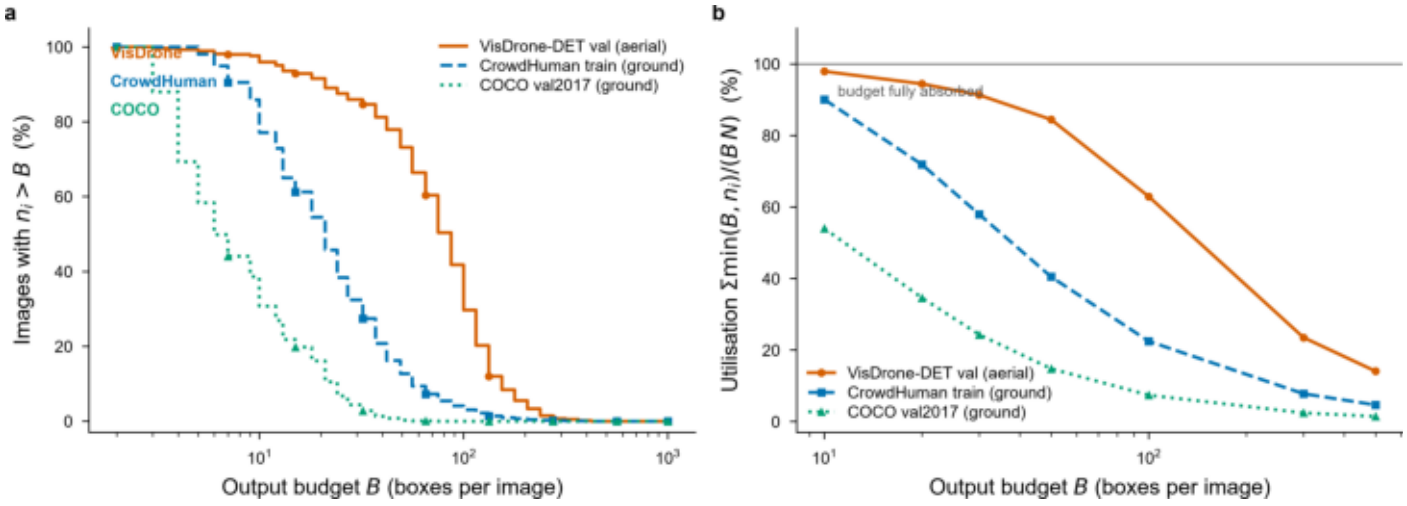


Figure 1. Per-image annotated object counts across detection benchmarks. a, Percentage of images whose object count n_i exceeds a budget B. b, Fraction of the permitted budget the annotations can absorb, Σ_i min(B, n_i)/(B·N). Colours, line styles and marker shapes all distinguish the curves, which remain separable in grayscale. N = 548 (VisDrone val), 15,000 (CrowdHuman train), 4,952 (COCO val2017). These are exact label counts, so no error bars are shown.

Table 1. Output-budget utilisation across benchmark splits.

The last three columns fix B = 100. ceiling is Σ_i min(B,n_i)/Σ_i n_i, the largest fraction of all annotated objects that an average per-image budget of B can in principle recover.

Split	Domain	Images (with gt)	mean n_i	max n_i	ceiling	util	frac(>100)
COCO val2017	ground	4,952	7.43	63	1.000	0.074	0.000
COCO train2017	ground	117,266	7.33	93	1.000	0.073	0.000
CrowdHuman train	ground	15,000	23.53	391	0.957	0.225	0.021
VisDrone test-dev	aerial	1,610	46.65	461	0.896	0.418	0.079
VisDrone val	aerial	548	70.73	317	0.890	0.630	0.195

At B = 100, utilisation spans 0.074 to 0.630 and the fraction of images on which the budget binds spans 0.000 to 0.195. The ordering follows the mean object count, not the domain. The largest object count does not predict utilisation either. CrowdHuman contains images with 391 annotated people, more than any VisDrone validation image (317), yet only 2.1% of CrowdHuman images exceed a budget of 100, against 19.5% of VisDrone validation images.

Two cautions apply. CrowdHuman flags ignore regions per box; excluding them lowers the mean object count by 3.8% and frac(n_i > 100) from 0.021 to 0.018, leaving the ordering unchanged. And Table 1 bounds the room a reallocation has; whether the worst-image recall changes depends on the detector, not on the labels, and the budget sweep below separates the two.

R1. Reallocation is positional: worst-image recall rises while mean per-image recall falls

To test whether the budget is better distributed than by a uniform constant, we trained YOLO26n on VisDrone2019-DET and evaluated on the 548-image validation split, for three seeds. For a fixed total budget S = ρ·Σ_i n_i with ρ = 0.5, we compared three allocations of the same total: a uniform quota q_i = min(c, n_i, p_i) with c chosen so that Σ_i min(c, n_i, p_i) = S; a quota proportional to the true object count (an oracle, see Methods), q_i = min(⌈a·n_i⌉, n_i, p_i) with a the largest multiplier whose spend stays within S; and an adaptive variant that caps each quota at p_i alone rather than at min(n_i, p_i). Here p_i is the number of boxes the detector can emit for image i, which caps the quota. Quotas are integers, and all three allocations spend within 0.02% of S.

Table 2. Allocation of a fixed output budget at ρ = 0.5, VisDrone2019-DET validation (N = 548).

Worst-image recall is min_i TP_i/n_i; mean per-image recall is the unweighted mean of TP_i/n_i over images. Cells are ordered one-to-many head (first entry) / one-to-one head (second entry); within each entry, seeds 42, 43 and 44 follow in that order. CI is a paired bootstrap over images, 10,000 resamples for every seed, with RNG seed 20261014.

Quantity	Uniform	Proportional	Difference (pt)
Worst recall, o2m	11.67 / 11.99 / 11.99%	20.00 / 20.37 / 20.00%	+8.33 / +8.38 / +8.01
Worst recall, o2o	11.36 / 11.67 / 11.36%	16.67 / 20.00 / 22.22%	+5.31 / +8.33 / +10.87
Total recall (micro), o2m	40.68 / 40.75 / 41.07%	42.06 / 42.00 / 42.30%	+1.38 / +1.25 / +1.23
Mean recall (macro), o2m	50.21 / 50.45 / 50.90%	43.85 / 43.88 / 44.03%	−6.36 / −6.57 / −6.87
Mean recall (macro), o2o	49.03 / 49.32 / 49.80%	42.89 / 43.15 / 43.42%	−6.14 / −6.17 / −6.38
95% CI for gain, o2m	n/a	n/a	[+5.86,+13.01] / [+5.63,+13.01] / [+5.86,+13.33]
95% CI for gain, o2o	n/a	n/a	[+1.93,+11.18] / [+5.26,+11.18] / [+7.48,+14.13]
P(gain > 0), o2m	n/a	n/a	1.0000 / 1.0000 / 1.0000
P(gain > 0), o2o	n/a	n/a	0.9975 / 1.0000 / 1.0000

On validation, proportional allocation raises worst-image recall by 5.31 to 10.87 points across the three seeds and both heads (Figure 2), and Figure 3 resolves the change image by image. Replacing the label count with the detector's own prediction count as the allocation signal recovers part of this gain at about half the mean cost (Methods; Supplementary Table S2). The gain is positive in every resample on five of the six seed-by-head conditions and in 99.75% on the sixth, the one-to-one head at seed 42. It is not a general improvement: mean per-image recall falls by 6.14 to 6.87 points on the same comparisons, so the intervention raises the minimum by lowering the average. Total recall is largely unchanged, with the two allocations differing by 1.23 to 1.40 points. When the budget binds on every image, the number of boxes that can be spent is fixed at $\Sigma_i \min(k_i, n_i) = \Sigma_i k_i = S$ by the budget rather than by the allocation. That identity constrains boxes emitted and not objects recovered: the false-positive share of the spent boxes rises with the budget, from under 1% at two boxes per image to over 40% at seventy, so the identity bounds the total recall instead of fixing it. Reallocating a fixed budget redistributes recall; total recall rises by only 1.2 to 1.4 points, which the equivalence test does not resolve.

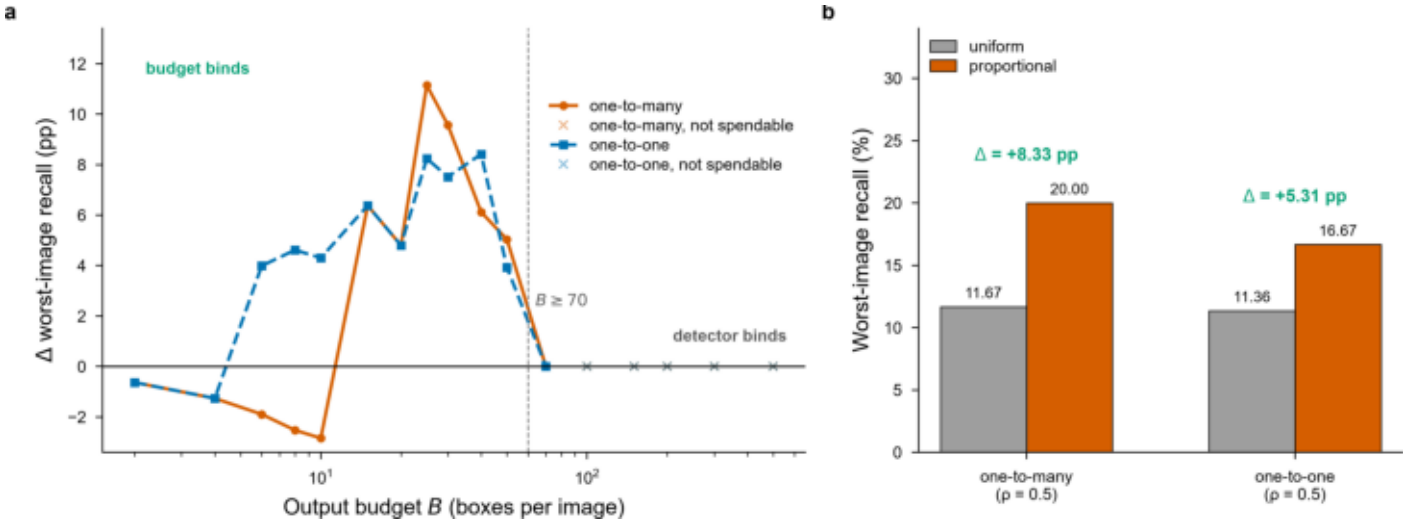


Figure 2. Gain in worst-image recall from reallocating a fixed output budget, in percentage points. a, Gain against budget B for both prediction heads; faint crosses mark budgets above which the budget cannot be spent in full, so the two allocations coincide and the comparison is an identity rather than a measurement. The horizontal axis is the average per-image budget. b, The same comparison at the pre-registered budget fraction ρ = 0.5, shown separately for the two prediction heads, so that its horizontal axis is the head and not the budget; only the extreme budgets of panel a are budget separations, and this panel is not one of them. N = 548 throughout; these are exact counts, so no error bars are shown.

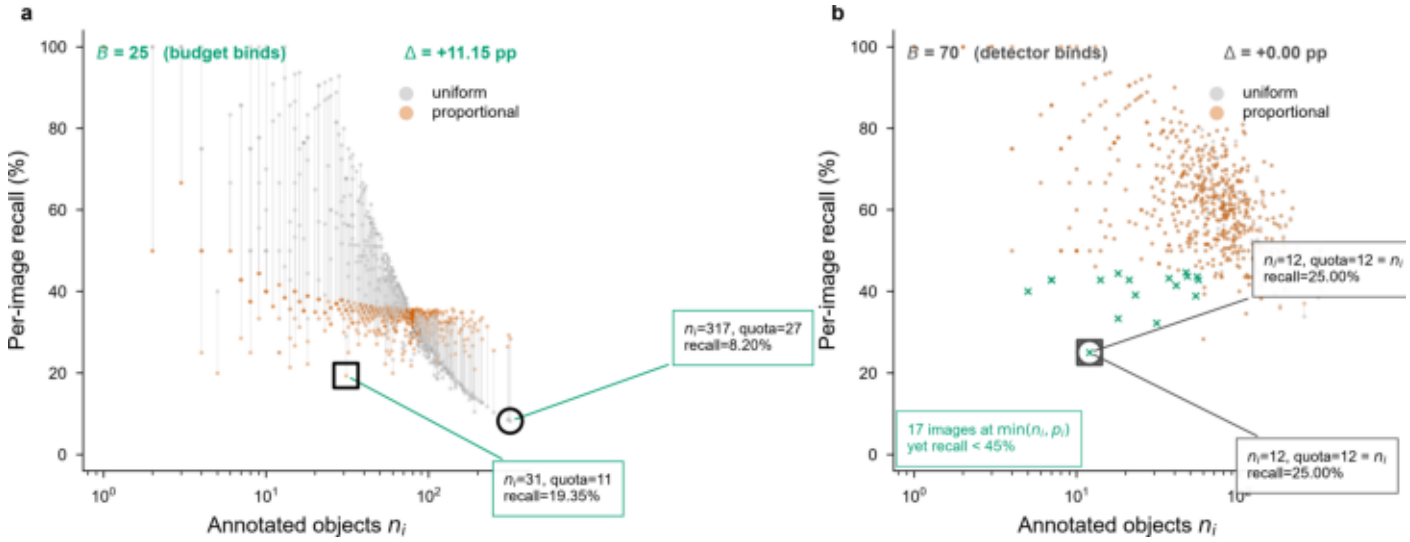


Figure 3. What the reallocation does at the level of individual images. a, b, Per-image recall against object count n_i under a uniform quota (grey) and proportional allocation (vermillion) at B = 25 and B = 70. Connectors join the two allocations for images whose recall changes by more than two points. Circles and squares mark the worst image under each allocation, labelled with its object count, quota and recall; a coloured outline means its quota reached the imposed label-count cap n_i, not that the detector ran out of predictions. (The per-image ceiling min(n_i, p_i) equals n_i throughout these records, because a prediction count p_i below the annotated count n_i never occurs; the outline condition is therefore the binding one.) Green crosses in b mark images that reached n_i under both allocations yet stay below 45% recall, which is why, under this label-count-capped convention, the gain there is exactly zero. N = 548 in both panels.

The one-to-many gain moves from +11.15 to +8.78 points and the one-to-one gain from +8.24 to +6.40 at B = 25, while at B = 50 the one-to-one gain moves in the opposite direction, from +3.93 to +7.65. The direction of the effect is stable, but its magnitude is not resolvable at the precision a single seed would suggest, which is why we report ranges rather than point estimates throughout, and why the confidence intervals extend past the threshold we pre-registered.

R2. The cost of the reallocation falls on the sparsest images

Because the total box count is conserved, only the distributional consequences matter. Table 3 breaks the per-image recall change down by object-count band, for the one-to-many head at ρ = 0.5 (Figure 4).

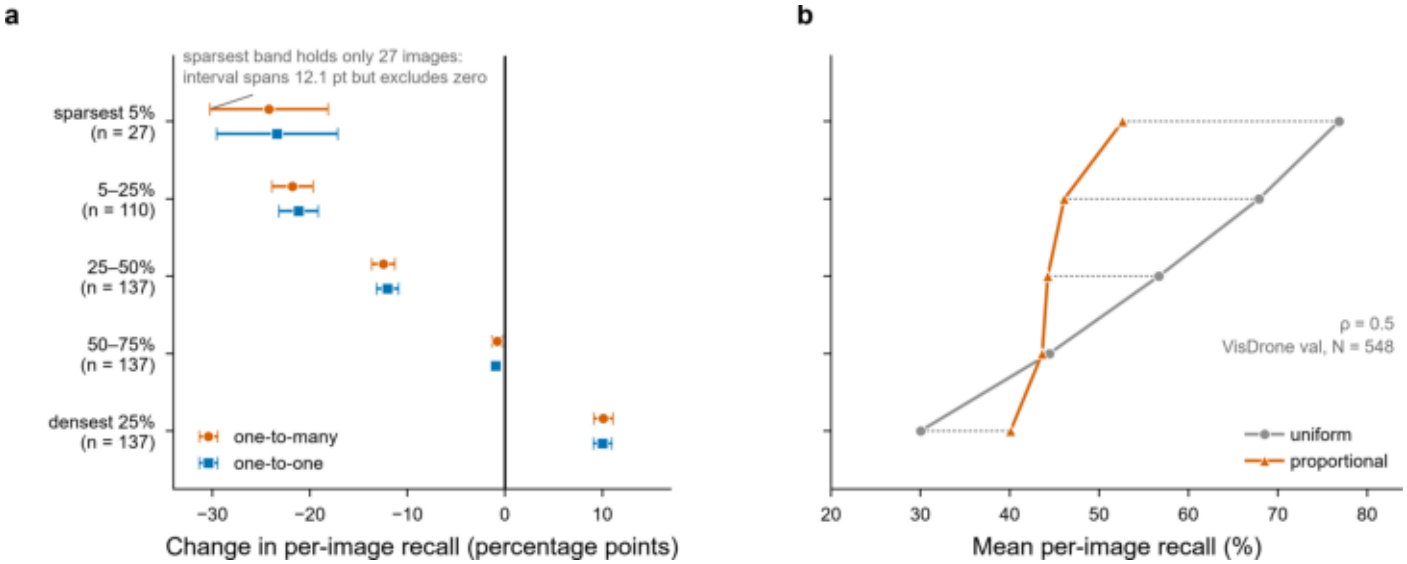


Figure 4. The cost of the reallocation by object-count band. a, Change in per-image recall, proportional minus uniform, in percentage points, for each n_i band; horizontal bars are 95% confidence intervals from a paired percentile bootstrap over images within the band (10,000 resamples; the RNG is derived per seed so that re-running one seed alone reproduces its row), and the vertical line marks zero change. Because the band boundaries are fixed by the label distribution, the interval estimates the mean cost of a band and not which images fall in it. Image counts per band are printed beside the labels. Colours and symbols follow Figure 2: circles for the one-to-many head, squares for the one-to-one head. b, Mean per-image recall under each allocation for the same bands; dashed connectors join the two allocations within a band, so the horizontal length of a connector is the change plotted in a. Grey circles mark the uniform allocation and vermillion triangles the proportional one. ρ = 0.5, VisDrone validation, N = 548. Panel b shows the one-to-many head only; the one-to-one head appears in a.

Table 3. Per-image recall by object-count band, VisDrone2019-DET validation, one-to-many head, seed 42.

ρ = 0.5. Δ is proportional minus uniform, in points. The band boundaries are fixed by the label distribution, which does not depend on the model. The interval is a paired percentile bootstrap over images within the band (10,000 resamples; the RNG is derived per seed so that re-running one seed alone reproduces its row), so it estimates the mean cost of the band and not the membership of the band. The one-to-one head gives the same ordering with slightly smaller magnitudes: −23.4 and +10.0 points at the two extremes, both intervals excluding zero.

n_i band	Images	n_i range	Uniform	Proportional	Δ	95% interval for Δ
sparsest 5%	27	1–9	76.85%	52.64%	−24.20	[−30.28,−18.14]
5–25%	110	10–39	67.92%	46.11%	−21.81	[−23.92,−19.69]
25–50%	137	40–65	56.70%	44.26%	−12.44	[−13.65,−11.28]
50–75%	137	65–93	44.44%	43.64%	−0.80	[−1.35,−0.26]
densest 25%	137	93–317	30.03%	40.10%	+10.07	[+9.11,+11.07]

The gain is concentrated in the densest quartile (+10.07 points) and the loss in the sparsest images (−24.20 points). The sparsest band holds 27 images, so its interval is widest at 12.14 points, yet still excludes zero: the cost is present in every one of the 10,000 resamples. Every band's interval excludes zero, and the interval ordering matches the band ordering, so under this scheme the gradient is not an artefact of band membership. In VisDrone, sparse images are those containing few and comparatively large objects: mean box area is 2.31×10^{-2} of the frame in the sparsest 5% against 1.18×10^{-3} in the densest quartile, and the Spearman correlation between object count and mean box area is −0.46 (Figure 5). We deliberately do not infer object distance from this, because the benchmark provides no depth annotation and the size–distance relation in aerial imagery also depends on camera tilt. Optimising worst-image recall therefore raises the recall of the dense images, which start from the lower baseline, and lowers that of the sparse images, which start from the higher baseline. Maximising the minimum is not the same objective as improving the worst case in the sense that matters for deployment.

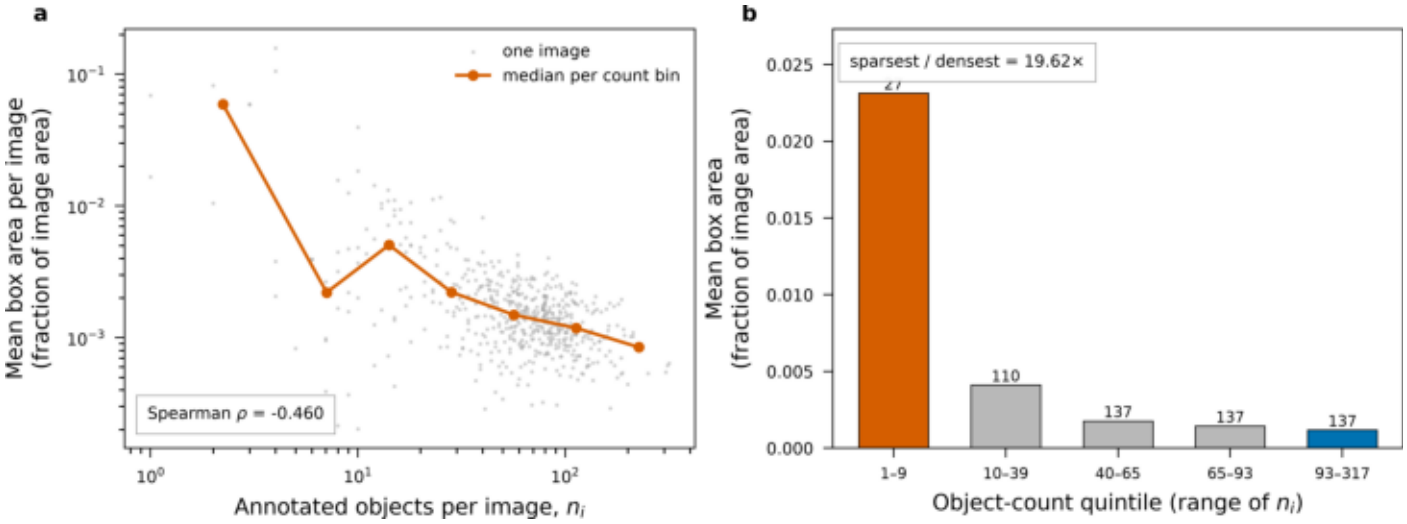


Figure 5. Sparse images hold larger objects. a, Mean annotation-box area per image against the annotated object count n_i, with the median per logarithmic count bin overlaid. b, Mean box area for the sparsest 5% of images (27 images, n_i 1-9) and the densest 25% (137 images, n_i 93-317), the same bands used in Figure 4; the number above each bar is the image count in that band. Areas are normalised (YOLO w*h, as a fraction of image area), so no image dimension enters the statistic. The Spearman correlation between object count and mean box area is -0.4596, and the sparsest band's mean box area is 19.62 times the densest band's. This is why a per-image quota costs the sparse images most: when an image holds few but large objects, each box carries more of that image's recall, so a proportional allocation that reclaims that image's boxes removes more recall than the same number of boxes taken from a dense image with small objects. We deliberately do not infer object distance, because VisDrone carries no depth annotation.

R3. The gain depends on the budget fraction and does not clear the pre-registered threshold uniformly

We pre-registered a decision rule before measuring anything: the allocation line would be treated as a standalone contribution if the worst-image gain reached 10 points, as a component of a larger study between 5 and 10 points, and abandoned below 5 points. The rule used the absolute point gain rather than a ratio, because a ratio would have required a denominator that a real detector cannot attain.

For seed 42, at the pre-registered budget fraction ρ = 0.5 the point estimates are +8.33 points on the one-to-many head and +5.31 on the one-to-one head, both inside the middle band, and the 95% percentile intervals are [+5.86, +13.01] and [+1.93, +11.18] (Table 2). Both intervals extend past the 10-point threshold at their upper limits, and neither is bounded away from it, so the point estimate alone would understate the uncertainty. The measured effect is therefore too uncertain to decide whether the allocation is worth reporting as a standalone result, and we report it as a component while stating the uncertainty explicitly.

A label-free top-up reproduces the gain (Table S6), and it is not an artefact of the budget fraction, but neither is it uniformly below the threshold; removing the label-count cap raises it further (Supplementary Table S5). Sweeping ρ over {0.30, 0.40, 0.50, 0.60, 0.70} and both heads gives ten conditions spanning +4.25 to +10.21 points. Nine of the ten fall in the middle band; one condition reaches the standalone band: the one-to-many head at ρ = 0.40 gives +10.21 points, which the pre-registered rule scores as A. The remaining condition, the one-to-one head at ρ = 0.70, falls below 5 points and scores as C. The full sweep is reported in Supplementary Table S1 and plotted against ρ in Supplementary Fig. S2; we do not report the pre-registered value alone, and we do not claim that the threshold is never reached.

R3b. The effect is hump-shaped in the budget

Table 1 reports how much room a fixed budget leaves. That is a property of the labels; whether reallocating changes the worst-image recall is a property of the detector, and the two come apart. To separate them we swept the average per-image budget B from 2 to 500, holding the total spend identical across the two allocations at every budget (relative difference between spends below 10^-4). Panel a of Figure 2 plots the same gain against the budget. Of the 17 budgets, 12 are spendable: at B ≥ 100 neither allocation can spend the budget, because the label-count cap saturates every quota, and those budgets are excluded from the spendable sweep.

Table 4. Worst-image gain from proportional allocation as a function of the budget.
VisDrone2019-DET validation, N = 548. S = B·N is the total budget. Of the 17 swept budgets, the twelve with B ≤ 70 are spendable (spend/S = 1.00000); this table lists ten of them plus B = 100 and B = 300, where the budget can no longer be spent in full. util is the label-side utilisation from Table 1. Gains are in points; each Δ column lists seeds 42, 43, 44, and Δ micro is on the same allocation as the gain.

B	util	spend / S	Δ worst o2m	Δ worst o2o	Δ micro o2m	Δ micro o2o
2	0.998	1.00000	−0.63 / −0.63 / −0.63	−0.63 / −0.63 / −0.56	−0.01 / +0.00 / −0.01	+0.01 / +0.01 / +0.01
6	0.990	1.00000	−1.89 / −1.89 / −1.89	+3.99 / +3.66 / −1.89	+0.10 / +0.08 / +0.09	+0.14 / +0.15 / +0.12
10	0.980	1.00000	−2.84 / −2.84 / −3.15	+4.30 / +6.11 / −3.15	+0.31 / +0.32 / +0.27	+0.34 / +0.33 / +0.31
15	0.963	1.00000	+6.38 / +6.38 / +9.40	+6.38 / +6.21 / +6.06	+0.50 / +0.47 / +0.44	+0.60 / +0.56 / +0.54
20	0.945	1.00000	+4.80 / +4.80 / +8.29	+4.80 / +7.98 / +7.98	+0.86 / +0.70 / +0.75	+0.81 / +0.79 / +0.85
25	0.929	1.00000	+11.15 / +8.78 / +11.48	+8.24 / +6.40 / +6.40	+1.00 / +0.96 / +1.08	+1.11 / +1.10 / +1.07
30	0.914	1.00000	+9.58 / +6.89 / +10.22	+7.52 / +4.82 / +4.51	+1.11 / +1.07 / +1.09	+1.21 / +1.11 / +1.19
40	0.882	1.00000	+6.12 / +8.03 / +6.12	+8.42 / +6.43 / +11.44	+1.23 / +1.10 / +1.11	+1.31 / +1.22 / +1.18
50	0.845	1.00000	+5.04 / +6.37 / +2.02	+3.93 / +7.65 / +8.69	+1.00 / +1.17 / +1.13	+1.17 / +1.11 / +1.00
70	0.762	1.00000	+0.00 / +0.00 / +0.00	+0.00 / +0.00 / +0.00	+0.01 / +0.02 / +0.01	+0.02 / +0.00 / +0.02
100	0.630	0.70728	+0.00 / +0.00 / +0.00	+0.00 / +0.00 / +0.00	+0.00 / +0.00 / +0.00	+0.00 / +0.00 / +0.00
300	0.236	0.23576	+0.00 / +0.00 / +0.00	+0.00 / +0.00 / +0.00	+0.00 / +0.00 / +0.00	+0.00 / +0.00 / +0.00

Three regimes appear, and we separate them by cause rather than by sign. At the smallest budgets the gain is negative on the one-to-many head, because proportional rounding gives an image with two to twenty objects one box where uniform gives two, so sparse images lose recall before dense ones gain. At intermediate budgets the gain peaks, at 11.15 points when B = 25, and the between-seed spread is largest there: the one-to-one head moves from +8.42 at B = 40 to +3.93 at B = 50, a range of 4.5 points across budgets and 4.8 points across seeds at fixed budget. At B ≥ 70 the gain is exactly zero, but for two different reasons: at B = 70 the two quota vectors differ while the worst image is the same image at its cap under both, whereas at every budget from B = 100 to 500 — five values in all — the two vectors are equal entry by entry and the comparison is an identity. Across the sweep the total recall moves by at most 1.31 points, and across the budget fraction sweep by at most 1.40 points. The full ρ sweep is reported in Supplementary Table S1.

R4. A reproducible YOLO26n baseline on VisDrone2019-DET

The detector used throughout is YOLO26n [16], trained from COCO weights [1] on the VisDrone2019-DET training split [15] for 100 epochs at 640×640 with batch size 8 on a single 8 GB consumer GPU. For seed 42, peak validation mAP50 is 0.3436 at epoch 83 and peak mAP50-95 is 0.1890 at epoch 82; the final-epoch values are 0.3385 and 0.1854. For seed 43 the two peaks coincide at epoch 80, at 0.3407 and 0.1858, with final-epoch values of 0.3365 and 0.1839. Seed 44 peaks at epoch 82, at 0.3421 and 0.1874, with final-epoch values of 0.3364 and 0.1838. The three seeds land within 0.3 points on mAP50 and 0.4 on mAP50-95, small relative to the between-seed spread of the allocation gain in R5b, and consistent with the effect being a property of how quotas interact with the label distribution rather than of detector quality. Training takes 238 s per epoch. We report no comparison against a published YOLO26n baseline: those figures used a different batch size and no stated seed, so the comparison would not be controlled. The per-epoch curve, the environment lock, and the weight hashes are released with the code, and the curves for all three seeds are plotted in Supplementary Fig. S1.

R5. The gain is positional, and part of the zero-gain regime is structural

Table 1 bounds the room a reallocation has. Whether it is worth using that room depends on the detector, and on which image the minimum lands on. Table 4 reports the measured gain; Table 5

reports the two quantities that decide it: whether the two quota vectors coincide, and the identity of the worst image under each allocation.

Table 5. Structural audit of the budget sweep, on all three seeds and both heads.
The measured gain itself is in Table 4; this table reports when the comparison is well-posed. identical vectors marks budgets at which the uniform and proportional quota vectors are equal entry by entry, so the comparison is an identity rather than a measurement. worst image at cap marks budgets at which the same image is the worst under both allocations. degenerate marks budgets at which some image's quota falls to one box, so its recall is decided by whether the single top prediction is correct.

B	identical vectors	worst image at cap (uniform / prop.)	degenerate
2	no	no / no	yes
6	no	no / no	yes
10	no	no / no	yes
15	no	no / no	yes
20	no	no / no	yes
25	no	no / no	yes
30	no	no / no	yes
40	no	no / no	no
50	no	no / no	no
70	no	yes / yes	no
100	yes	yes / yes	no
300	yes	yes / yes	no

Three features of Table 5 matter. At B = 100 and 300 the comparison is vacuous: the two quota vectors are equal entry by entry, which we verify by exact array comparison rather than by inspecting the gain, so both allocations collapse to spending every box the detector can emit and the measured zero is an identity. At B = 70 the vectors differ on 314 of 548 images, yet the worst image is the same image under both allocations (index 22, with 12 annotated objects) and has reached its cap under both, so its recall is literally the same number and the minimum is taken over one unchanged value. Below B = 70 the gain is real and sign-varying: the largest is +11.15 points at B = 25, the smallest is −2.84 at B = 10, and the sign is not uniform across heads at B = 6 and 10.

The gain comes from the identity of the worst image changing. At B = 25 under a uniform quota the worst image is the densest one, with 317 annotated objects, held to 8.20% recall; redistribution moves quota to it, and the minimum is then taken over a different image at 19.36%. Some images gain and others lose (densest quartile +10.07 points, sparsest 5% -24.20); what changes is which image is worst. This is why the gain disappears once no image's quota is binding, and why the intervention should not be described as an improvement of a fixed worst case.

R5b. What replicates across seeds, and what does not

Two things replicate exactly and one does not. The allocation-side quantities are fixed by the quota vector, which depends on n_i and the budget and not on the network weights: at B = 25 the uniform worst-image recall is 11.67% for the one-to-many head on seed 42 and 11.99% on seed 43, and 11.36% and 11.67% respectively for the one-to-one head. The two heads are not interchangeable here. The same holds at B ≤ 15, where quotas of one or two boxes quantise per-image recall to multiples of 1/n_i and the gains are identical across seeds, because the detector's ordering no longer decides the minimum.

The zero-gain regime also replicates exactly, for the structural reasons given above: at B = 100 and 300 the two quota vectors are equal on all three seeds and both heads, and at B = 70 the worst image is the same one at its cap in all four combinations.

The magnitudes do not replicate, and we report them as ranges. The between-seed spread reaches 4.8 points on the one-to-one head, where the gain at B = 50 runs from +3.93 on seed 42 to +8.69 on seed 44, and at B = 25 both heads lose 2 to 3 points from the first seed to the second. Every magnitude in this paper is therefore a range over three seeds and two heads, and the ranges are wide enough that a single-seed comparison at a single budget could be made to show almost any effect within them.

Discussion

This work began as an attempt to predict, per image, whether an output budget should be reallocated. The prediction failed and we withdrew it. What survived is a measurement of what reallocation buys, what it costs, and when the question is vacuous. We report both halves, because the failure is informative about the shape of the problem.

The failed attempt is worth stating precisely, because the same construction is natural. We tried to decide, per image, whether the budget or the detector limited that image, by comparing the worst image's quota against the number of boxes it could physically emit. The quantity is computable from stored predictions and needs no retraining. It is not diagnostic, for two reasons. First, shuffling the detector's confidence-ordered hit sequences across images, which holds the label counts and the budget fixed and substitutes a different detector, leaves the verdict unchanged in 198 of 200 draws while the measured gain becomes non-positive in every one of them: the quantity tracks the labels, not the detector. Second, at seed 42 on the one-to-one head, ten non-degenerate conditions at budgets 58 to 67 predict a gain that is measured negative, down to −2.8 points; a counterexample, not a rate. The deeper reason is that the number of predictions a detector emits never falls below the number of annotated objects on the same image, so the prediction count cancels and the construction reduces to a statement about the labels alone, the same class of quantity as the utilisation statistics we already argued are uninformative. The only reliable test of such a quantity is to change the detector and see whether the verdict follows.

Under a uniform quota the densest image is pinned as the worst case, and once the quota is redistributed that image is no longer the worst, so the minimum is taken over a different image. Describing this as an improvement of the worst case would be arithmetically accurate and substantively misleading, because the binding constraint moved rather than relaxed.

The zero-sum property follows from the same accounting, with one correction we owe the reader. When the budget binds on every image, the conservation identity fixes the spend. That identity is about boxes emitted, not about objects recovered: the false-positive share of the spent boxes rises with the budget, so the identity bounds the total recall rather than fixing it. Measured total recall changes by at most 1.41 points across the twelve spendable budget conditions, consistent with that bound. Reallocation is therefore not a performance intervention. It moves recall between images, and the images it moves recall away from are the sparsest ones, which in VisDrone contain the fewest and largest objects. A practitioner who adopts worst-image recall as an objective is, in this benchmark, choosing to degrade the images that are easiest to resolve and thereby improve the ones that are hardest to count. That is a defensible choice for some applications and a poor one for others, but it should be made explicitly rather than inherited from a metric name.

The gain is bounded by the nature of the worst image. Because it replaces rather than improves that image, it helps only when a quota can help it. On validation the worst image is quota-limited, so the proportional allocation moves the minimum elsewhere. On test-dev both allocations share the same worst image: the uniform quota already covers its whole capacity of 8 boxes and recovers none (Supplementary Table S3), while the vectors differ on 1568 of 1610 images and both spend 37,551 boxes. The comparison is well posed and the gain is +0.000 on both heads. The mechanism replicates while the aggregate statistic does not.

Several limitations bound these claims. The results come from three training seeds of one detector on one benchmark; the between-seed spread is material, reaching 3.0 points on one head, so the magnitudes we report are ranges rather than points, including the 24.2-point loss on the sparsest images, whose interval spans 12.14 points and still excludes zero, and the 11.2-point peak gain. Above a per-image budget of about 100 the budget can no longer be spent in full, because the total number of boxes the detector can emit is smaller than the budget; for those conditions the two allocations coincide exactly, and we report them as a structural identity rather than as evidence about allocation. CrowdHuman [17] object counts depend on an annotation convention, with ignored regions optionally excluded; we report both versions, and the cross-benchmark ordering is unchanged. Finally, the two heads we test are two settings of the same network rather than two independent implementations, so the measurement rests on one detector family.

Two directions follow. The measurement should be repeated on detectors whose per-image recall does not rise with object count, since the positional gain we observe is a property of the detector's failure profile rather than of the allocation. And the objective deserves scrutiny: worst-image recall is an extreme order statistic, so its sampling variability is governed by a handful of images, which is why the intervals we report are wide relative to the effect. Lower quantiles of the per-image recall distribution would trade the interpretability of a worst case for estimates a finite benchmark can actually resolve.

Methods

Detector and training

We used YOLO26n as implemented in Ultralytics 8.4.177. YOLO26 is an end-to-end detector with two parallel heads over the same backbone: a one-to-many head that produces dense predictions and requires non-maximum suppression, and a one-to-one head that produces a fixed set of predictions and requires none. The one-to-one head is the head used at deployment; we report both, since the choice of head changes the allocation result by 3.0 points.

Training used the Ultralytics default optimiser with mosaic augmentation at 1.0, disabled for the final 10 epochs, translation 0.1, scale 0.5, horizontal flip 0.5, and HSV jitter (h 0.015, s 0.7, v 0.4). Mixup, copy-paste, and rotation were disabled to keep the recipe comparable to the published VisDrone anchor. Early stopping was disabled so that the full 100-epoch curve is available. Data-loader workers were set to 0, which is required on Windows: with workers > 0 the validation pass deadlocks during worker spawn.

Evaluation protocol

Detection quality is reported as AP@[.50:.05:.95] following the VisDrone protocol. The validation split was used for all development; the test-dev split was reserved for a single evaluation at freeze time, run through the same label-informed pipeline. Predictions were generated at confidence 0.001 with IoU 0.7, matching the COCO and LVIS sweeps.

For the budget experiments, the detector is run once with max_det set to the largest budget in the sweep. Smaller budgets are then obtained by truncating each image's predictions to its top q_i by confidence. This is exact rather than approximate: we verified on 40 images across eight budgets and both heads that truncating a large-max_det run reproduces running at the small max_det directly, with 40/40 images matching and a maximum coordinate deviation of 0.0. Because the equivalence holds, each budget costs no additional inference.

A full-resolution pass over all 548 validation images at once exhausts GPU memory, so the scan is run in chunks that are merged afterwards; the merge is checked against annotation counts recomputed from the label files, and any mismatch aborts the merge.

Allocation

For a total budget S = ρ·Σ_i n_i, the uniform allocation gives every image the same quota c, chosen by bisection as the largest value for which Σ_i min(c, n_i, p_i) ≤ S. The proportional allocation is defined through the budget rather than directly: we bisect the multiplier a and give image i the quota min(⌈a·n_i⌉, n_i, p_i) for the largest a whose total spend Σ_i min(q_i, n_i, p_i) does not exceed S. Writing the quota directly as min(⌈ρ·n_i⌉, n_i, p_i) would overspend, because the ceiling is applied per image; on the validation split at ρ = 0.5 it exceeds S by 0.70% and changes 135 of 548 quotas. Quotas are rounded up to integers, and the spend of an allocation is Σ_i min(q_i, n_i, p_i). Every quota is capped at min(n_i, p_i) by construction: the p_i term because the detector cannot emit more boxes than it predicts, and the n_i term because this paper studies the label-count-capped regime, in which each image may be allocated at most as many boxes as it has annotated objects. That second cap is a modelling choice and not a property of the detector: because predictions are ranked by confidence, a box beyond the first n_i can still be the first to match a previously unmatched object, so the cap removes output slots that could in principle raise recall. It binds on no reported condition with B ≤ 70, where every quota stays at or below the annotated count, and it is what saturates the allocation at B ≥ 100; the saturation there is therefore label-count saturation, not exhaustion of the detector's prediction capacity. Hence q_i ≤ min(n_i, p_i) holds for every image at every budget we report, so this spend equals the number of boxes the detector actually emits, Σ_i min(q_i, p_i); we checked the two sums agree exactly at every swept budget, on both heads, with a maximum difference of zero boxes., where p_i is the number of predictions the detector emitted for image i. The three allocations are compared only when their spends agree to within 1%. Because n_i is the annotated count, the proportional allocation is an oracle: it measures what redistribution could buy if each image's object count were known in advance, not a deployable policy.

The proportional rule is the classical max-min fair rule for the total-spend constraint, and we use it as a baseline rather than as an optimum of the recall objective: which image ends up worst depends on where the detector's errors fall, not only on how many boxes each image receives. That maximisation is a classical max-min fair allocation problem, and progressive water-filling is its standard solution [13]; we do not claim the optimisation as a new result. What we report is its behaviour when the constraint is the number of boxes a detector may emit, and the empirical consequences of that setting, which neither the optimisation literature nor the detection-evaluation literature addresses.

Prediction-proportional allocation

The proportional allocation above uses the annotated object count n_i, which is not available at inference time; it therefore measures what redistribution could buy if each image's object count were known in advance, and it is an oracle rather than a policy. To ask what the proportional rule would do if the allocation signal were the detector's own output rather than the label count, we replace n_i with the number of predictions the detector emits for that image once, at the largest budget in the sweep, p_i. This signal is available at inference; the top-up below is not, so the resulting pipeline is label-informed throughout and is not a deployable policy. The same bisection, the same integer rounding, and the same top-up are applied, so the only change is the allocation signal.

On the one-to-many head the prediction-proportional variant recovers 53% to 78% of the oracle gain in worst-image recall (+5.72, +6.55 and +4.24 points for seeds 42, 43 and 44, against +8.33, +8.38 and +8.01 for the oracle), and it does so at roughly half the macro cost (−3.44, −3.49 and −3.83 points against −6.36, −6.57 and −6.87). On the one-to-one head it recovers almost none of the gain (+0.32 points), which is what the positional account predicts: on that head the worst image is already close to its ceiling under the uniform quota, so reallocation has little to replace. Because p_i exceeds n_i on every image in our records, the prediction-based signal is a noisy proxy for the oracle one, and the gap between the two rows is the price of not knowing the object count.

Budget convention and top-up

Two conventions are easy to misread, so we state them explicitly. First, B is the total budget divided by the number of images, S = B·N. It is an average per-image budget, not a cap on any single image: the uniform allocation fixes one quota c for every image, so an image whose capacity exceeds c can never receive more, but the proportional allocation can give an image several times B when that image is dense. At B = 70 on the validation split the largest quota is 206 boxes under the uniform allocation and 312 under the proportional one, against an average of 70. Second, integer bisection alone usually underspends S by a few boxes. We therefore add a deterministic top-up: while the spend is below S, one box is added to the image whose next prediction carries the highest marginal true-positive gain. That gain is read off the label matches, so the top-up is label-informed; it is applied identically to every allocation, which keeps the comparison internally valid, but it means no allocation in this paper is a deployable policy. The top-up changes the quota of 0.4% of images at B = 2, 8.9% at B = 25 and 1.1% at B = 70; it is applied identically to every allocation, so it cannot favour one over another. Without it the two spends would differ and the comparison would confound where boxes go with how many are spent.

Top-up ablation

The main text reports the oracle top-up, which reads true-positive matches from the annotations. To test whether the reported gain depends on that choice, we repeated every ρ condition with three label-free variants: no top-up at all; a largest-remainder rule that gives the deficit to the images with the largest fractional part of their unrounded quota; and a capacity rule that gives it to the images with the most remaining prediction slots. The last two need only the annotation-free quantities already used to build the quota. With the budget matched to within 0.01%, both label-free rules reproduce the worst-image gain to within 0.32 points of the oracle arm, and the largest-remainder rule exceeds it at one condition by 3.33 points. The no-top-up arm under-spends by up to 2.75% at the smallest budget fraction and is therefore not a valid control there; where its spend does match, it agrees with the oracle arm exactly. The reported gain therefore does not depend on label access in the top-up step. Per-condition values are in Supplementary Table S6.

Pre-registration and decision rule

The decision rule, the threshold values, the budget fraction ρ = 0.5, and the statistic were fixed before the detector was evaluated. The rule was revised once, before any measurement, because the first version used a ratio whose denominator, the capacity-only ceiling S/Σ_i n_i, is unattainable by a real detector; using it would have made both the accept and the abandon thresholds unreachable. The revision to an absolute point gain is recorded, with the reason, in the repository. The implementation carries a self-test that constructs synthetic worlds with known outcomes and asserts that the rule separates them; it currently passes all five assertions.

Test-set evaluation and label dependence

The test-dev split was evaluated once, after the method and every decision rule had been frozen. That single evaluation is nonetheless not an independent test of a deployable policy: the deterministic top-up step used on test-dev is the same one used on validation, and it reads true-positive matches from the test-dev annotations to decide which image receives the next box. Every allocation analysed here, on both splits, is therefore an oracle-style diagnostic rather than a deployable procedure. What the test-dev result establishes is narrower and still informative: when the worst image is detection-limited rather than quota-limited, reallocating a fixed budget leaves it at zero recall even though the quota vectors differ on 1568 of 1,610 images.

Sensitivity to the label-count cap

Because the n_i term is a modelling choice rather than a detector property, we repeated the whole

ρ sweep with that term removed: quotas were re-derived with only the prediction cap p_i, spend was counted as the boxes actually emitted, Σ_i min(q_i, p_i), and true positives were still capped at n_i because no image can yield more. Removing the label-count cap raises the worst-image gain at every one of the ten conditions, by 0.63 to 3.78 points, and raises the micro-recall change by 1.80 to 4.49 points; the per-condition values are in Supplementary Table S5 and in results/tables/gt-cap-sensitivity.json. The main text reports the capped numbers, so every gain quoted there is a conservative estimate specific to the label-count-capped convention, and the positional conclusion does not depend on that cap.

Statistical analysis

Uncertainty in the worst-image gain is estimated by a paired bootstrap over images, 10,000 resamples, with the pairing unit being the image rather than the prediction [18]. Within each resample both allocations are recomputed from the resampled image set and the change is evaluated on that same set, so the comparison is never made across different image sets. The resampling seed is fixed at 20261014. We report the 95% percentile interval and the fraction of resamples with a positive change.

For the ten ρ-sweep conditions we also report a family-wise correction rather than asserting that it is unnecessary. Testing the one-sided null that the worst-image gain is not positive, per condition, gives raw p values between 0.0001 and 0.0054 across the ten conditions, with 0.0054 the largest of the ten; Holm step-down [19] at a family-wise α of 0.05 leaves the smallest Holm threshold at 0.0050 and the largest at 0.0500, and all ten remain rejected — the smallest raw value clears the first step's threshold of 0.005, and the ordering then carries the rest. The correction therefore changes none of the decisions we report, and the quantities are in results/tables/fwer-and-equivalence.json, and the per-condition ranks, Holm thresholds and adjusted *p* values are in Supplementary Table S4.

The claim that reallocating leaves total recall approximately unchanged is a claim of equivalence, not of a failure to detect a difference, and we test it as such. The equivalence margin is pre-registered at 1.0 percentage point (src/eval/stats.py, rationale in docs/decisions.md); it was fixed before these measurements and is not chosen to fit them. Across the ten conditions the point estimate of that change ranges from 0.74 to 1.40 points. Against the pre-registered 1.0-point margin none of the ten passes: the two one-sided tests give p between 0.33 and 0.73, far above 0.05, because four of the ten point estimates already exceed the margin. We therefore do not claim equivalence at that margin and report the observed range instead. Had the test statistic been scaled wrongly, it would have returned p near zero and a spurious equivalence claim; the margin is in points, so the per-image contributions enter the test in those units.

Two cautions apply to the interpretation. The bootstrap quantifies sampling variability over images; it does not capture variability over training runs, and the results here come from three seeds. And the worst-image recall is an extreme order statistic, so it is governed by the few images with the largest or smallest object counts. Both cautions are the reason the confidence intervals in Table 2 extend past the pre-registered threshold.

Image and annotation counts

The Images column of Table 1 counts images that carry at least one annotated object, which is the population any recall statistic is defined over. COCO val2017 and train2017 contain a further 48 and 1,021 images with no annotations; these enter no recall and change no reported fraction, so they are excluded from N and from every per-image table in this paper.

Data

VisDrone2019-DET is a publicly available aerial detection benchmark with 6,471 training, 548 validation, and 1,610 test-dev images across ten classes, distributed under CC BY-NC-SA 3.0 for

academic use. Its per-image object counts range from 1 to 461. COCO 2017 and CrowdHuman are used only through their annotation files, since Table 1 depends on label distributions alone; no images from either benchmark were downloaded, and no model was trained on them. CrowdHuman object counts use boxes tagged person, matching the default of the reference conversion script; ignored regions are excluded in a reported variant.

Code availability

All code, configurations, per-epoch training curves, per-image prediction records and the scripts that generate every figure and table are available at https://github.com/xyx-cmd-maker/yolo26-output-budget (release tag v1.0-submission, MIT licence). The repository also ships a single entry point, scripts/check_all.py, which reruns every consistency and numerical check used here, including independent recomputation of the reported statistics from the stored records. The datasets are not redistributed there and remain under their own licences.

Code and reproducibility

Training, evaluation, and every analysis in this paper run through a single entry point per stage. Each run writes a code fingerprint (SHA-256 over the source tree), the git commit, the full argument set, and the weight hashes. The dependency set is pinned in requirements.lock.txt, generated by pip freeze in the environment that produced the results. Configurations for the reported runs are stored in the repository at https://github.com/xyx-cmd-maker/yolo26-output-budget/tree/v1.0-submission. The self-tests for the statistical routines and for the decision rule are part of the test suite, and the numerical self-checks we ran against independent recomputation are recorded in Supplementary Note S1. A large language model was used to assist with code implementation, literature search, and language editing; the research question, the experimental design, the allocations compared, and all numerical results and conclusions are the authors' own, and every reported number is recomputed from stored prediction records by the released scripts.

Data and code availability

VisDrone2019-DET is public under CC BY-NC-SA 3.0 for academic use. COCO 2017 and CrowdHuman are public as well; only their annotation files were used here, and neither their images nor any model trained on them contributes to a result. Code, configurations, per-epoch curves, per-image prediction records and the figures are released with the manuscript. Because the per-image records hold the full confidence-ordered prediction list for every validation image, every number, table and figure can be recomputed without retraining or re-inference.

References

1. Lin, Tsung-Yi, Maire, Michael, Belongie, Serge, Hays, James, Perona, Pietro, Ramanan, Deva, Dollár, Piotr, & Zitnick, C. Lawrence (2014). Microsoft COCO: Common Objects in Context. ECCV. 10.1007/978-3-319-10602-1_48

2. Gupta, Agrim, Dollar, Piotr, & Girshick, Ross (2019). LVIS: A Dataset for Large Vocabulary Instance Segmentation. CVPR. 10.1109/CVPR.2019.00550

3. Dave, Achal, Dollár, Piotr, Ramanan, Deva, Kirillov, Alexander, & Girshick, Ross (2021). Evaluating Large-Vocabulary Object Detectors: The Devil is in the Details. arXiv preprint. arXiv:2102.01066

4. Fu, WeiFu, Nie, CongChong, Sun, Ting, Liu, Jun, Zhang, TianLiang, & Liu, Yong (2021). LVIS Challenge Track Technical Report 1st Place Solution: Distribution Balanced and Boundary Refinement for Large Vocabulary Instance Segmentation. arXiv preprint. arXiv:2111.02668

5. Oksuz, Kemal, Cam, Baris Can, Kalkan, Sinan, & Akbas, Emre (2022). One Metric to Measure them All: Localisation Recall Precision (LRP) for Evaluating Visual Detection Tasks. IEEE TPAMI. 10.1109/TPAMI.2021.3130188

6. Ultralytics (2026). Platform xView dataset makes most targets tiny and caps validation recall at 23.6%. GitHub issue, Ultralytics repository. https://github.com/ultralytics/ultralytics/issues/25992

7. Xiong, Z., Zeng, Z., Chen, Y., Xu, S., Yan, Y., & Qi, D. (2026). DenseScout: Algorithm-System Co-design for Budgeted

Tiny Object Selection on Edge Platforms. ACM Multimedia 2026 (accepted). arXiv:2604.25300

8. Pham Cong, H. & Luong, H. V. D. (2026). The Decision Value of Perception Compute. arXiv preprint. arXiv:2609.35910

9. Feng, Junyi, Hua, Jiashen, Lai, Baisheng, Huang, Jianqiang, Li, Xi, & Hua, Xian-sheng (2020). Learning to Generate Content-Aware Dynamic Detectors. arXiv preprint. arXiv:2012.04265

10. Liang, Feng, Lin, Chen, Guo, Ronghao, Sun, Ming, Wu, Wei, Yan, Junjie, & Ouyang, Wanli (2020). Computation Reallocation for Object Detection. ICLR 2020. arXiv:1912.11234

11. Wang, Zhaohui (2026). The Recall Ceiling of LLM Recommendation Reranking. arXiv preprint; stated CIKM '26. arXiv:2609.27953

12. Buckley, Chris & Voorhees, Ellen M. (2004). Retrieval Evaluation with Incomplete Information. SIGIR. 10.1145/1008992.1009000

13. Bertsekas & Gallager (1992). Data Networks (2nd ed.). Prentice Hall. ISBN 978-0132009241

14. Hardy, Littlewood, & Polya (1934). Inequalities. Cambridge University Press. ISBN 978-0521358806

15. Zhu, Pengfei, Wen, Longyin, Du, Dawei, Bian, Xiao, Fan, Heng, Hu, Qinghua, & Ling, Haibin (2022). Detection and Tracking Meet Drones Challenge. IEEE TPAMI. 10.1109/TPAMI.2021.3119563

16. Jocher, Glenn, Qiu, Jing, Liu, Mengyu, Lyu, Shuai, Akyon, Fatih Cagatay, & Kalfaoglu, Muhammet Esat (2026). Ultralytics YOLO26: Unified Real-Time End-to-End Vision Models. arXiv preprint. arXiv:2606.03748

17. Shao, Shuai, Zhao, Zijian, Li, Boxun, Xiao, Tete, Yu, Gang, Zhang, Xiangyu, & Sun, Jian (2018). CrowdHuman: A Benchmark for Detecting Human in a Crowd. arXiv preprint. arXiv:1805.00123

18. Efron & Tibshirani (1993). An Introduction to the Bootstrap. Chapman & Hall. ISBN 978-0412042317

19. Holm (1979). A Simple Sequentially Rejective Multiple Test Procedure. Scand. J. Stat. 6(2), 65-70. JSTOR:4615733

Declarations

Author contributions

Y.X. designed the study, implemented the training and evaluation pipeline, produced all per-image prediction records and the numerical results reported here, and drafted the manuscript. Z.Z. supervised the work, ensured compliance with academic standards, provided scientific advice throughout, and commented on successive drafts. Both authors analysed and interpreted the results, revised the manuscript critically, and approved the submitted version.

Competing interests

The authors declare no competing interests.

Funding

This research received no specific grant from any funding agency in the public, commercial, or not-for-profit sectors.

Ethics

No human or animal subjects were involved, and no personally identifiable information was processed. CrowdHuman images were not downloaded; only its annotation file was used, to compute a label distribution.

Data availability

VisDrone2019-DET is public under CC BY-NC-SA 3.0 for academic use. COCO 2017 and CrowdHuman are public as well; only their annotation files were used here. Code, configurations, per-epoch curves, per-image prediction records and all figures are archived at https://github.com/xyx-cmd-maker/yolo26-output-budget (release v1.0-submission, MIT licence); the datasets are not redistributed there.